From Satellite to Screen

Google Earth Engine ➔ Python ➔ Heat Map Animation

NASA MODIS Terra MOD11A1 v061 | 2001–2023

STAGE	TOOL	OUTPUT
1. Source	NASA / MODIS satellite	Raw thermal imagery
2. Cloud store	Google Earth Engine	Image collection in GEE
3. Process	GEE JavaScript editor	Annual mean LST images
4. Export	GEE Export to Drive	LST_land_YYYY.tif files
5. Download	Google Drive	25 .tif files on your PC
6. Visualise	Python (this script)	GIF + MP4 animation

UNDERSTANDING THE DATA

What is MODIS?

MODIS stands for **Moderate Resolution Imaging Spectroradiometer**. It is a sensor mounted on NASA's Terra satellite, launched in December 1999. The satellite passes over the entire Earth's surface every 1–2 days, measuring reflected and emitted radiation in 36 wavelength bands.

What is MOD11A1?

MOD11A1 is the specific MODIS product used here. Each letter and number means something:

- **MOD** — Terra satellite (as opposed to MYD which is Aqua satellite)
- **11** — Land Surface Temperature and Emissivity product
- **A** — daily global acquisition
- **1** — 1 km spatial resolution
- **v061** — version 6.1 (the latest reprocessed, most accurate version)

What does the data actually measure?

Land Surface Temperature (LST) is NOT air temperature. It is the temperature of the physical surface of the land as seen from space — what you would measure if you put a thermometer directly on the ground or road. LST can be 10–20°C hotter than air temperature on a sunny day.

What is a .tif file here?

A GeoTIFF (.tif) is not a normal photo. It is a **grid of numbers** where each cell = one temperature value for one patch of land. Metadata inside the file tells software exactly where on Earth each cell sits.

```
Normal photo pixel:  [R=255, G=128, B=0]   -> orange color
LST GeoTIFF pixel:   [31.4]                 -> 31.4 degrees Celsius

Grid size example:
  4320 columns = 360 degrees longitude / 0.0833 deg per pixel
  1680 rows    = 140 degrees latitude  / 0.0833 deg per pixel
  = ~1 km resolution at equator
```

■ **One .tif file = one year of annual mean LST for the entire land surface of Earth.**

GOOGLE EARTH ENGINE (GEE)

Google Earth Engine is a cloud platform that stores decades of satellite data and lets you process it without downloading anything first. Instead of downloading 25 years of daily MODIS tiles (terabytes), you write JavaScript in the GEE browser editor and GEE runs the computation on Google's servers. You only download the final result.

STEP 1 Open GEE Code Editor

code.earthengine.google.com

Go to **code.earthengine.google.com** and sign in with a Google account that has been approved for GEE access (free for research/non-commercial use). You will see a JavaScript editor in the browser.

STEP 2 Load the MODIS Collection

ImageCollection in GEE

In GEE, all satellite data is stored as an **ImageCollection** — a stack of images over time. You filter it to the years and band you need.

```
// Load MODIS MOD11A1 v061 — daily LST global
var dataset = ee.ImageCollection('MODIS/061/MOD11A1')
    .filterDate('2001-01-01', '2001-12-31')  // one year
    .select('LST_Day_1km');                   // daytime LST band

// Result: a stack of ~300 daily images for 2001
// Each image = one day's worth of LST readings globally
```

STEP 3 Convert Units and Mask Clouds

Scale factor + QC band

Raw MODIS LST values are stored in Kelvin multiplied by 0.02 (to fit large numbers into small integers). You must convert to Celsius:

```
// MODIS stores LST as: raw_value * 0.02 = Kelvin
// To convert to Celsius: (raw * 0.02) - 273.15

function convertLST(image) {
    var lst_celsius = image
        .multiply(0.02)      // apply scale factor
        .subtract(273.15)    // Kelvin to Celsius
        .rename('LST_C');    // rename the band
    return lst_celsius;
}

var lst = dataset.map(convertLST);
```

Note: If you skip the scale factor, your temperatures will be around 6000°C. This is the most common GEE beginner mistake with MODIS LST.

STEP 4 Compute Annual Mean

ImageCollection.mean()

Reduce the ~300 daily images for one year into a single annual mean image. GEE does this on its servers — no downloading involved yet.

```
// Collapse all daily images into one annual mean
var annualMean = lst.mean();

// annualMean is now a single image where
```

```
// each pixel = average LST for that year

// Clip to land only (remove ocean pixels)
var landMask = ee.Image('MODIS/006/MOD44W/2015_01_01')
    .select('water_mask').eq(0);

var lstLand = annualMean.updateMask(landMask);
```

EXPORTING FROM GEE

STEP 5 Export to Google Drive

Export.image.toDrive()

Once the annual mean image is computed, export it as a GeoTIFF to your Google Drive. GEE runs this as a background task.

```
// Export one year
Export.image.toDrive({
    image: lstLand,
    description: 'LST_land_2001',    // task name in GEE
    fileNamePrefix: 'LST_land_2001', // filename on Drive
    folder: 'LST_Export',            // Drive folder
    scale: 1000,                     // 1 km per pixel
    region: ee.Geometry.Rectangle(
        [-180, -60, 180, 80]),       // global extent
    crs: 'EPSG:4326',               // flat lat/lon grid
    maxPixels: 1e13,                 // allow large export
    fileFormat: 'GeoTIFF'
});
```

After clicking Run, go to the **Tasks** tab in GEE and click **Run** next to your task. GEE will process and upload the file to your Google Drive. Takes 5–20 minutes per year depending on server load.

STEP 6 Repeat for All 25 Years

Loop in GEE

Instead of repeating Step 5 manually 25 times, use a loop in GEE:

```
var years = ee.List.sequence(2001, 2023);

years.evaluate(function(yearList) {
    yearList.forEach(function(year) {

        var start = ee.Date.fromYMD(year, 1, 1);
        var end   = ee.Date.fromYMD(year, 12, 31);

        var annual = ee.ImageCollection('MODIS/061/MOD11A1')
            .filterDate(start, end)
            .select('LST_Day_1km')
            .map(function(img) {
                return img.multiply(0.02)
                          .subtract(273.15);
            })
            .mean()
            .updateMask(landMask);

        Export.image.toDrive({
            image: annual,
            description: 'LST_land_' + year,
            fileNamePrefix: 'LST_land_' + year,
            folder: 'LST_Export',
            scale: 1000,
            region: ee.Geometry.Rectangle(
                [-180, -60, 180, 80]),
```

```
            crs: 'EPSG:4326',
            maxPixels: 1e13
        });
    });
});
```

Note: This creates 23 export tasks in the Tasks tab. You must click Run on each one. GEE does not auto-run them for safety.

STEP 7 Download from Google Drive

Drive to your PC

After all tasks complete (green tick in GEE Tasks tab), go to Google Drive and download the entire LST_Export folder. You will have files named exactly:

```
LST_land_2001.tif
LST_land_2002.tif
LST_land_2003.tif
...
LST_land_2023.tif

Place all files in one folder:
C:\Users\Lenovo\Desktop\Heat action day\
```

■ Filename format matters. The Python script reads the year by splitting on underscore. LST_land_2001.tif -> split('_') -> ['LST','land','2001.tif'] -> year=2001.

PYTHON SIDE — What Happens to the .tif Files

STEP 8 **Find All Files** glob + os

```
import glob, os

DATA_DIR  = r'C:\Users\Lenovo\Desktop\Heat action day'
tif_files = sorted(glob.glob(
    os.path.join(DATA_DIR, 'LST_land_*.tif')
))

# Result:
# ['...\LST_land_2001.tif',
#  '...\LST_land_2002.tif', ...]
```

STEP 9 **Read Each .tif as a Grid of Numbers** GDAL

```
from osgeo import gdal
import numpy as np

ds     = gdal.Open('LST_land_2015.tif', gdal.GA_ReadOnly)
band   = ds.GetRasterBand(1)
data   = band.ReadAsArray().astype(np.float32)
nodata = band.GetNoDataValue()   # the 'empty' sentinel value
ds     = None

# Clean up
data[data == nodata] = np.nan    # no data -> NaN
data[data < -90]     = np.nan    # impossible values
data[data >  70]     = np.nan

# data.shape is e.g. (1680, 4320)
# data[500, 1000] = 28.4  <- one pixel, one temperature
```

STEP 10 **Draw the Map** Matplotlib + Cartopy

```
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature
from matplotlib.colors import Normalize

fig = plt.figure(figsize=(14, 7.5), facecolor='black')
ax  = fig.add_axes([0.03, 0.08, 0.82, 0.83],
                   projection=ccrs.PlateCarree())
ax.set_extent([-180, 180, -60, 80])

# draw ocean and borders
ax.add_feature(cfeature.OCEAN,     facecolor='#0a0a14')
ax.add_feature(cfeature.COASTLINE, edgecolor='#666666')
ax.add_feature(cfeature.BORDERS,   edgecolor='#444444')

# draw the temperature grid
```

```
norm = Normalize(vmin=-25, vmax=55)  # fixed for all years
ax.imshow(data, cmap=LST_CMAP, norm=norm,
          origin='upper',
          extent=[-180, 180, -60, 80])

# year label
ax.text(0.02, 0.95, '2015',
        transform=ax.transAxes,
        color='white', fontsize=44)
```

STEP 11

Capture Frame + Save Animation

BytesIO + imageio

```
import io
from PIL import Image
import imageio.v2 as imageio

# capture figure to memory
buf = io.BytesIO()
plt.savefig(buf, format='png', dpi=150, facecolor='black')
buf.seek(0)
frame = np.array(Image.open(buf))
plt.close(fig)

# after all years are rendered:
imageio.mimsave('output.gif', frames, fps=2, loop=0)
```

FULL PIPELINE — ONE PAGE

Everything from satellite to screen, in order:

#	Where	What happens	Input	Output
1	NASA / Space	Terra satellite measures thermal emission from land	Sunlight + land	Raw digital counts
2	NASA servers	Counts converted to temperature (Kelvin)	Raw counts	MOD11A1 daily images
3	GEE storage	25 years of daily images stored in ImageCollection	MOD11A1 archive	ee.ImageCollection
4	GEE editor	Filter by year, convert to Celsius	ImageCollection	Annual LST images
5	GEE editor	Compute annual mean, mask ocean pixels	Daily images	1 image per year
6	GEE Tasks	Export as GeoTIFF to Google Drive	GEE image	LST_land_YYYY.tif
7	Google Drive	Download files to your PC	Drive files	25 .tif files
8	Python / GDAL	Open .tif, read as 2D NumPy array	.tif file	data[rows, cols]
9	Python / NumPy	Replace nodata, clean bad values	Raw array	Clean float array
10	Python / Matplotlib	Draw world map with temperature colors	NumPy array	Matplotlib figure
11	Python / imageio	Capture frame, append to list	Figure	numpy frame
12	Python / imageio	Save all frames as GIF and MP4	frames list	output.gif / .mp4

■ The split: GEE handles the satellite data and computation (Steps 1–7). Python handles the visualisation (Steps 8–12). The .tif file is the handoff point between the two.

Why not do everything in GEE?

- GEE can visualise maps but cannot make custom animations with glowing text, custom colormaps, north arrows, scale bars, or MP4/GIF exports.
- GEE is JavaScript — Python has more powerful visualisation libraries.
- Downloading the processed .tif files gives you permanent local copies you can re-visualise anytime without re-running GEE.

Why not do everything in Python?

- 25 years of daily MODIS data is terabytes. You cannot download that to a laptop.

- GEE runs the mean computation on Google's infrastructure in minutes. Doing it locally would take days.
- GEE already has MODIS data indexed and ready. Downloading raw tiles and assembling them yourself is complex.

KEY CONCEPTS TO REMEMBER

EPSG:4326 — What it means

When you export from GEE with crs: 'EPSG:4326', you are choosing a **flat lat/lon grid** where every pixel covers the same number of degrees but NOT the same real-world area. A pixel at 60°N covers roughly half the land area of a pixel at the equator. This is why the Python script applies a cosine(latitude) weight when computing the global mean — to correct for this distortion.

```
# Pixels look equal on screen but cover different real areas:

Equator (0 deg):   1 pixel = ~1 km x 1 km  (full area)
60 deg N/S:        1 pixel = ~1 km x 0.5 km (half area)
80 deg N/S:        1 pixel = ~1 km x 0.17 km (tiny area)

# Correction: weight each pixel by cos(latitude)
weights = np.cos(np.deg2rad(lats))  # 1.0 at equator, ~0 at poles
```

Scale Factor — The Most Important GEE Detail

MODIS stores LST as integers to save space. The real value is always **raw_integer × 0.02**. Forgetting this gives physically impossible temperatures.

```
Raw stored value:  14950
After scale:       14950 * 0.02 = 299.0 Kelvin
After conversion:  299.0 - 273.15 = 25.85 Celsius  <- correct

If you forget scale: 14950 - 273.15 = 14676 Celsius <- wrong!
```

NoData Values

Pixels with no valid observation (ocean, persistent cloud cover, polar night) are stored as a special number — typically **-9999** or the band's fill value. GDAL reports this via GetNoDataValue(). Replace with NaN so they are invisible on the map.

```
nodata = band.GetNoDataValue()    # e.g. -9999.0
data[data == nodata] = np.nan     # invisible on map

# NaN pixels show as transparent -> ocean color shows through
# That is why ocean appears dark blue, not as a temperature
```

Fixed Color Scale Across All Frames

The colormap range (vmin=-25, vmax=55) must be identical for every year. If it changes per year, a frame showing 2001 and a frame showing 2016 will use different color meanings — the animation becomes misleading.

```
# CORRECT — fixed across all 23 frames
norm = Normalize(vmin=-25, vmax=55)

# WRONG — changes per year, makes animation lie
norm = Normalize(vmin=data.min(), vmax=data.max())
```

Why 2000 Was Excluded

MODIS Terra launched December 1999. The 2001 data year is the first full calendar year. Year 2000 is incomplete (only ~10 months of data) and has early-orbit sensor calibration issues. Its area-weighted mean (+23.00°C) is ~1.5°C above all other years and never returns — a clear artifact.

```
EXCLUDE_YEARS = {2000}

active_files = [
    f for f in tif_files
    if int(os.path.basename(f)
           .split('_')[2].split('.')[0])
       not in EXCLUDE_YEARS
]
```

COMMON MISTAKES AND HOW TO AVOID THEM

Mistake	What happens	Fix
Forget scale factor (×0.02) in GEE	Temperatures show as 6000°C+	Always: img.multiply(0.02) .subtract(273.15)
Don't mask ocean in GEE	Ocean pixels show as temperature values	Apply land mask: .updateMask(landMask)
Don't replace nodata with NaN in Python	-9999 values show as extreme cold (dark blue)	data[data==nodata] = np.nan
Per-year vmin/vmax in colormap	Animation misleading — colors mean different things	Use fixed Normalize (vmin=-25, vmax=55)
Include year 2000	+23°C spike makes trend look like sudden warming	EXCLUDE_YEARS = {2000}
Don't close GDAL dataset (ds = None)	File stays locked, memory leak on long runs	Always set ds = None after ReadAsArray()
origin='lower' in imshow	Map is upside down (Antarctica at top)	Always use origin='upper'
Forget buf.seek(0) after savefig	Empty frame — image reads from end of buffer	Always buf.seek(0) before Image.open(buf)